

Urban Simulation Platform Projects

Energy System Optimization Design

Written by: Alireza Adli

Project Reseacher: Navid Shirzadi

Supervisor: Professor Ursula Eicker

Project Integrator: Alireza Adli

The project is a part of The NGCI's Urban Simulation Platform project. The platform has integrated the application of NGCI's research projects in Python programming language. These projects are developed by graduate students and researchers of the institute towards sustainable urban development in six main areas: energy systems, building, transportation, vegetation, waste and recycling, and liveability.

The integration of a project refers to further developing a research project following the coding style of the platform in the Object Oriented Programming (OOP) paradigm (if applicable). This is done to employ multiple projects in a single workflow.

For more information about this piece of software and documentation, contact Alireza Adli (alireza.adli@concordia.ca)
For detailed technical information on the project, read Navid Shirzadi PhD thesis report and the original code documentation that he has provided (the link is available in Introduction).

Contents

1	Introduction	3
2	The Solver License	4
2.1	Installing The Scip Solver and Running The Project	4
3	Input Data	7
4	Output Data	8

1 Introduction

The project is a workflow that returns the optimized Energy System configuration of a building. The documentation on the original code (developed and written by Navid Shirzadi) is available on Microsoft Teams through the below folder:

Teams → CERC NextGenCities Platform → Documents → Energy System Optimization → Energy System Optimization Design Manual_Navid Shirzadi.pdf

The current document discusses the refactored code for integrating the project into the CERC's Urban Simulation Platform. This includes information on the solver license (section The Solver License), how to run the project using an API call (in section Installing The Scip Solver and Running The Project), names of the constants, variables and parameters in the refactored code (sections Input Data and Output Data).

The project was completed in January 2023. The name of the module is `energy_optimization_po.py` and is available through the below link:

https://rs-loy-gitlab.concordia.ca/alireza.adli/navid_shirzadi_project

The Python module regarding the project's Application Programming Interface (API) has been attached to the same folder:

Teams → CERC NextGenCities Platform → Documents → Energy System Optimization → `energy_optimization_po_api.py`

The below steps have been taken in refactoring the project:

- Changing the name of all the Python variables following the snake case convention.
- Wrapping the project in a function.
- Redefining and changing the order of defining the model's functions to increase the level of readability.

- Applying a solver, licensed under the Apache 2.0 License
- Developing an API
- Defining default parameters so that the program works with or without user inputs
- Developing the documentation

2 The Solver License

`energy_optimization_po.py` uses a Python package, namely `pyomo`. `pyomo` needs a solver. Solver is the software that finds a solution to a specific problem, in this case, optimization. The solver that `pyomo` uses for optimization has been downloaded from:

<https://scipopt.org/index.php#download>

The applied version (8.0.3) is available for Windows, MacOS, Linux and Raspberry. It is licensed under Apache 2.0 License, enabling the platform to use and giving its users free access.

2.1 Installing The Scip Solver and Running The Project

The best approach for installing the solver is to make a Python virtual environment for the whole project and then install the solver in that environment. Because the solver may have dependency issues with other Python packages. Here is the instruction:

- Download the last update of the solver, which is licensed under the Apache 2.0 License (the link in the section The Solver License can be used).
- Install the Scip (different solvers will be extracted in the chosen destination directory as a result).

- Put the directory that keeps the solvers in Windows' Path through the Environment Variables (there are lots of resources available on the internet explaining how to do that).
- Make A Python virtual Environment for running the project
- Install the Python package using pip or Anaconda commands.

```
pip install pyscipopt
```

- Clone the project, including `energy_optimization_po.py` and the data folder, from the below link:
https://rs-loy-gitlab.concordia.ca/alireza.adli/navid_shirzadi_project
- Install the necessary Python packages for the project using pip or Anaconda commands.

For the project: `matplotlib.pyplot`, `pyomo`, `pandas`, `numpy`

For running the project from the API: `flask`, `flask_apispec`, `flask_restful`, `marshmallow`

- Put the API's Python module (`energy_optimization_po_api.py`) in the main folder of the project.
- Run the `energy_optimization_po_api.py`.
- Open a Git Bash application to test the program using a curl command. The project has been developed to work based on and without user inputs. Here are some examples of the curl commands to be run in the Git Bash application:

Example one:

```
1 curl -X POST -H "Content-Type: application/json" -d '{"  
    pv_cost": 1246, "grid_purchase_max": 200000, "  
    wind_turbine_operating_cost": 0.002, "  
    grid_purchase_price": 0.08, "grid_selling_price": 0.08,  
    "nonrenewable_grid_portion": 0, "area":24487.0}' http  
://localhost:5000/result
```

Example two:

```
1 curl -X POST -H "Content-Type: application/json" -d '{"  
    pv_cost": 1246, "area":24487.0}' http://localhost:5000/  
result
```

Example three:

```
1 curl -X POST -H "Content-Type: application/json" -d '{}'  
http://localhost:5000/result
```

- A group of values will be put out in the Git bash application as they can be returned to the front-end (object_fu, cost_of_energy, number_of_pv, number_of_wind_turbines, battery_storage_capacity, total_capital_cost, renewable_penetration, payback_period). And a CSV file (results_local_new.csv) will be output in the data folder.

With minor changes in the current API code, it can be associated with the gamification front-end or the CityLayer through the hub. The project can now work independently using its current API following the above instructions.

3 Input Data

In table 1, 'Values' refer to the Default Values in the code. PV and wind turbines Maintenance and operation costs have been shortened to PV and wind turbines operating_cost.

Table 1: Input parameters and their corresponding variables in the Python program.

Parameter (unit)	Python	Values
Battery Charging Eff (%)	battery_charging	0.95
Battery Discharging Eff (%)	battery_discharging	0.95
Max Charge Discharge Rate (%)	charge_dis_charge_max	0.35
Portion of Electricity that- can be sold to the grid (None)	electricity_to_grid	1
PV Cost (\$/kW)	pv_cost	1246
DC to DC Converter Cost (\$/kW)	dc_dc_converter_cost	100
Maximum Grid Purchase (kWh/day)	grid_purchase_max	200000
Wind Turbine Cost (\$/kW)	wind_turbine_cost	2077
Wind Turbine Operating Cost (%)	wind_turbine_operating_cost	0.002
Capital Recovery Factor (None)	capital_recovery_factor	0.064
Capital Recovery- Factor of Battery (None)	capital_recovery_factor_battery	1.7743
AC to DC Rectifier Cost (\$/kW)	ac_dc_rectifier_cost	100
DC to AC Inverter Cost (\$/kW)	dc_ac_inverter_cost	100
Battery Price per kWh (\$/kWh)	battery_kwh_price	670
Battery Operating Cost (\$/kWh.hr)	battery_operating_cost	0.00054
Maintenance Cost (\$/kWh)	battery_maintenance_cost	13.2
Battery Replacement Cost (\$/kWh)	battery_replacement_cost	670

Grid Purchase Price (\$/kWh)	grid_purchase_price	0.08
Grid Selling Price (\$/kWh)	grid_selling_price	0.08
Minimum State of- Charge of the Battery (%)	charge_state_min	0.2
Unmet Load Penalty- Coefficient (\$/kW)	loss_coefficient	0.8
Cost of Operation and- Maintenance of PV Panel (%)	pv_operating_cost	0.001
Portion of the Grid's- Non-Renewable Electricity ϵ (None)	nonrenewable_grid_portion	0
Environmental Coefficient- Penalty β (\$/kW)	environmental_coefficient_penalty	0.8

4 Output Data

Numerical values in table 2 refer to the upper limit of the same variable, which will be input as the input value. The model returns the optimum value for all the variables as output values.

Table 2: Output variables and their corresponding variables in the Python program.

Variables (unit)	Python	Values
PV Capacity (kW)	pv_panels_number	3800
Wind Turbine Capacity (kW)	wind_turbines_number	1200
Battery Capacity (kWh)	battery_capacity	None
Battery Energy Level (kWh)	battery_energy	None

Battery Charging Power (<i>kW</i>)	charging_capacity	None
Battery Discharging Power (<i>kW</i>)	discharging_capacity	None
Wind Used Power (<i>kW</i>)	wind_used_energy	None
Wind Surplus Power (<i>kW</i>)	wind_power_surplus	None
PV Used Power (<i>kW</i>)	pv_used_power	None
PV Surplus Power (<i>kW</i>)	pv_power_surplus	None
Grid Purchase (<i>kW</i>)	purchased_grid_capacity	2000
Grid Sell (<i>kW</i>)	sold_grid_capacity	2000
Unmet Load (<i>kW</i>)	loss	100
Binary Variable (None)	gamma	None
Binary Variable (None)	teta	None
Binary Variable (None)	lambdaa	None
Binary Variable (None)	eta	None
